

Practical No.9

Aim

To implement Hill Cipher and analyze its security.

Code

```
// Hill Cipher Implementation and Security Analysis - TypeScript
// Simple 2D Matrix operations for Hill Cipher
class Matrix {
    static multiply(matrix: number[][], vector: number[], mod: number): number[] {
        const rows = matrix.length;
        const cols = matrix[0].length;
        const result: number[] = [];

        for (let i = 0; i < rows; i++) {
            let sum = 0;
            for (let j = 0; j < cols; j++) {
                sum += matrix[i][j] * vector[j];
            }
            result.push(((sum % mod) + mod) % mod);
        }

        return result;
    }

    static determinant(matrix: number[][]): number {
        const n = matrix.length;
        if (n === 1) return matrix[0][0];
        if (n === 2) return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];

        // For larger matrices, use simple expansion (simplified)
        let det = 0;
        for (let j = 0; j < n; j++) {
            const minor = this.minor(matrix, 0, j);
            const sign = j % 2 === 0 ? 1 : -1;
            det += sign * matrix[0][j] * this.determinant(minor);
        }
    }
}
```

```
    return det;
}

static minor(matrix: number[][], row: number, col: number):
    return matrix
        .filter((_, i) => i !== row)
        .map(row => row.filter((_, j) => j !== col));
}

static adjugate(matrix: number[][]): number[][] {
    const n = matrix.length;
    if (n === 1) return [[1]];
    if (n === 2) {
        return [
            [matrix[1][1], -matrix[0][1]],
            [-matrix[1][0], matrix[0][0]]
        ];
    }
    // For larger matrices
    const adj: number[][] = [];
    for (let i = 0; i < n; i++) {
        adj[i] = [];
        for (let j = 0; j < n; j++) {
            const minor = this.minor(matrix, j, i);
            const sign = (i + j) % 2 === 0 ? 1 : -1;
            adj[i][j] = sign * this.determinant(minor);
        }
    }
    return adj;
}

static modInverse(value: number, mod: number): number {
    value = ((value % mod) + mod) % mod;

    // Find modular multiplicative inverse
    for (let i = 1; i < mod; i++) {
        if ((value * i) % mod === 1) {
            return i;
        }
    }
}
```

```
    }
  }
  throw new Error('No modular inverse exists');
}

static inverse(matrix: number[][], mod: number): number[][] {
  const det = this.determinant(matrix);
  const detMod = ((det % mod) + mod) % mod;

  // Check if invertible
  if (this.gcd(detMod, mod) !== 1) {
    throw new Error('Matrix not invertible mod ${mod}. gcd(');
  }

  const detInv = this.modInverse(detMod, mod);
  const adj = this.adjugate(matrix);

  // Modular inverse:  $\det^{-1} * \text{adj} \pmod{\text{mod}}$ 
  const inv: number[][] = [];
  for (let i = 0; i < adj.length; i++) {
    inv[i] = [];
    for (let j = 0; j < adj[0].length; j++) {
      const val = ((detInv * adj[i][j]) % mod + mod) % mod;
      inv[i][j] = val;
    }
  }

  return inv;
}

static gcd(a: number, b: number): number {
  a = Math.abs(a);
  b = Math.abs(b);

  while (b !== 0) {
    [a, b] = [b, a % b];
  }
  return a;
}
```

```
}

// Hill Cipher Implementation
class HillCipher {
  key: number[][];
  mod: number;
  det: number;

  constructor(keyMatrix: number[][], validate: boolean = false) {
    this.key = keyMatrix;
    this.mod = 26;
    this.det = Math.round(Matrix.determinant(keyMatrix));

    if (validate) {
      this.validateKey();
    }
  }

  validateKey(): void {
    const n = this.key.length;

    // Check if square
    for (let i = 0; i < n; i++) {
      if (this.key[i].length !== n) {
        throw new Error('Key matrix must be square');
      }
    }

    const detMod = ((this.det % this.mod) + this.mod) % this.mod;

    if (Matrix.gcd(detMod, this.mod) !== 1) {
      throw new Error(
        `Key matrix not invertible mod ${this.mod}. ` +
        `det = ${this.det}, gcd(det mod 26, 26) must be 1`
      );
    }
  }

  charToNum(char: string): number {

```

```
    return char.toLowerCase().charCodeAt(0) - 'a'.charCodeAt(0)
  }

  numToChar(num: number): string {
    return String.fromCharCode((num % this.mod) + this.mod)
  }

  textToVector(text: string): number[] {
    return text.split('').map(c => this.charToNum(c));
  }

  vectorToText(vector: number[]): string {
    return vector.map(v => this.numToChar(v)).join('');
  }

  encrypt(plaintext: string): string {
    plaintext = plaintext.toLowerCase().replace(/\s/g, '');
    const blockSize = this.key.length;

    // Pad plaintext if necessary
    while (plaintext.length % blockSize !== 0) {
      plaintext += 'x';
    }

    let ciphertext = '';

    for (let i = 0; i < plaintext.length; i += blockSize) {
      const block = plaintext.substring(i, i + blockSize);
      const vector = this.textToVector(block);
      const encrypted = Matrix.multiply(this.key, vector, this.mod);
      ciphertext += this.vectorToText(encrypted);
    }

    return ciphertext;
  }

  decrypt(ciphertext: string): string {
    const detMod = ((this.det % this.mod) + this.mod) % this.mod;
```

```
// Check invertibility before attempting decryption
if (Matrix.gcd(detMod, this.mod) !== 1) {
  throw new Error(
    `Cannot decrypt: Key matrix not invertible mod ${this.mod},
    `det = ${this.det}, det mod ${this.mod} = ${detMod},
    `gcd(${detMod}, ${this.mod}) = ${Matrix.gcd(detMod, this.mod)}
  );
}

ciphertext = ciphertext.toLowerCase().replace(/\s/g, '');
const blockSize = this.key.length;

// Calculate inverse key matrix
const keyInv = Matrix.inverse(this.key, this.mod);

let plaintext = '';

for (let i = 0; i < ciphertext.length; i += blockSize) {
  const block = ciphertext.substring(i, i + blockSize);
  const vector = this.textToVector(block);
  const decrypted = Matrix.multiply(keyInv, vector, this.mod);
  plaintext += this.vectorToText(decrypted);
}

return plaintext;
}
}

// Verify the given example: pay -> XNS
function verifyExample(): void {
  console.log('=' .repeat(60));
  console.log('VERIFICATION OF GIVEN EXAMPLE');
  console.log('=' .repeat(60));

  // Given key matrix
  const K: number[][] = [
    [17, 17, 25],
    [21, 18, 21],
    [2, 2, 19]
```

```
];

const cipher = new HillCipher(K, false);
const plaintext = 'pay';

console.log('\nKey Matrix K (mod 26):');
K.forEach(row => console.log(` ${row} `));

console.log('\nPlaintext: ${plaintext}');

// Show encryption process
console.log('\nEncryption Process:');
console.log(` p = 15 (a=0, b=1, ..., p=15)`);
console.log(` a = 0`);
console.log(` y = 24`);
console.log(` P = [15, 0, 24]^T`);

const ciphertext = cipher.encrypt(plaintext);
console.log('\nCiphertext: ${ciphertext.toUpperCase()}');

// Verify decryption (will fail for non-invertible key)
console.log('\nAttempting decryption...');
try {
    const decrypted = cipher.decrypt(ciphertext);
    console.log(`Decrypted: ${decrypted}`);
} catch (e) {
    console.log(`Decryption FAILED (expected): ${(e as Error)}`);
    console.log(`\n Note: This key matrix is NOT invertible`);
    console.log(` Encryption works, but decryption is`);
    console.log(` This demonstrates the IMPORTANCE of`);
}

if (ciphertext.toUpperCase() === 'XNS') {
    console.log('\n VERIFIED: \'pay\' encrypts to \'XNS\' as');
} else {
    console.log('\n MISMATCH: Expected \'XNS\', got \'${ciphertext}\'');
}
}
```

```
// Security analysis
function securityAnalysis(): void {
  console.log('\n' + '='.repeat(60));
  console.log('SECURITY ANALYSIS OF HILL CIPHER');
  console.log('='.repeat(60));

  console.log(`
1. BRUTE FORCE ATTACK ANALYSIS
-----

For an n×n Hill cipher:
- Key space: All invertible n×n matrices modulo 26
- Total n×n matrices: 26^(n²)

For n=3 (3×3 matrix):
- Total possible matrices: 26^9  5.4 × 10^12
- Invertible matrices: (26) × 26^(n²-n) × ...  1.6 × 10^12

Strength Assessment: MODERATE
  Better than simple substitution (26!  4×10^26 for n=1, but
  1.6 trillion keys is non-trivial but feasible with modern
  Special hardware can enumerate 10^12 keys in hours/days

Time estimates (assuming 10^9 key trials/second):
- n=2: ~0.16 seconds (very weak)
- n=3: ~26 minutes (weak)
- n=4: ~18 days (moderate)
- n=5: ~3.7 years (stronger)

2. CRYPTANALYSIS ATTACK ANALYSIS
-----

Known Plaintext Attack (THE CRITICAL WEAKNESS):

For an n×n Hill cipher with n known plaintext-ciphertext pairs

Step 1: Set up matrix equation
- P × K^T = C (mod 26)
- Where P is n×n plaintext matrix, C is n×n ciphertext matrix
- Solve: K^T = P^(-1) × C (mod 26)
```


Step 2: Compute key

- Calculate $P^{-1} \bmod 26$ (if invertible)
- Multiply by C to recover K^T
- Transpose to get K

Requirements for Attack:

- Need exactly n blocks of known plaintext-ciphertext pair
- Plaintext matrix must be invertible ($\det \neq 0 \bmod 26$)

Example: For 3×3 Hill cipher

- Need only 3 known pairs of 3-character blocks
- Solve using linear algebra
- Recover complete key matrix
- Break entire system

Strength Assessment: VERY WEAK

Known plaintext attack completely breaks the cipher

Only n blocks needed (e.g., 9 characters for 3×3)

No diffusion beyond block size

Linear structure is exploitable

Other Attacks:

- Chosen Plaintext: Trivial - choose identity matrix as plaintext
- Ciphertext Only: Frequency analysis on digraphs/trigraphs

3. COMPARATIVE ASSESSMENT

Hill Cipher vs Classical Ciphers:

+ Advantages:

Polygraphic - obscures single-letter frequencies

Mathematical elegance

Can use larger block sizes for diffusion

- Disadvantages:

Linear algebra - vulnerable to mathematical attacks

Known plaintext attack breaks it completely

No avalanche effect (small plaintext change = predictable)
Block structure preserves patterns

4. KEY WEAKNESSES SUMMARY

CRITICAL: The Hill cipher has FUNDAMENTAL weaknesses:

1. LINEARITY: Encryption is linear transformation
 - $C1 + C2 = \text{Encrypt}(P1 + P2)$
 - Exploitable through algebraic attacks
2. DETERMINISTIC: Same plaintext block = same ciphertext
 - No randomness or chaining between blocks
3. KNOWN PLAINTEXT RECOVERY:
 - n blocks $\rightarrow$ complete key recovery
 - $O(n^3)$ time complexity (matrix inversion)

```
`);  
}
```

```
// Main function  
function main(): void {  
    verifyExample();  
    securityAnalysis();  
}  
  
// Run if executed directly  
main();
```

Output

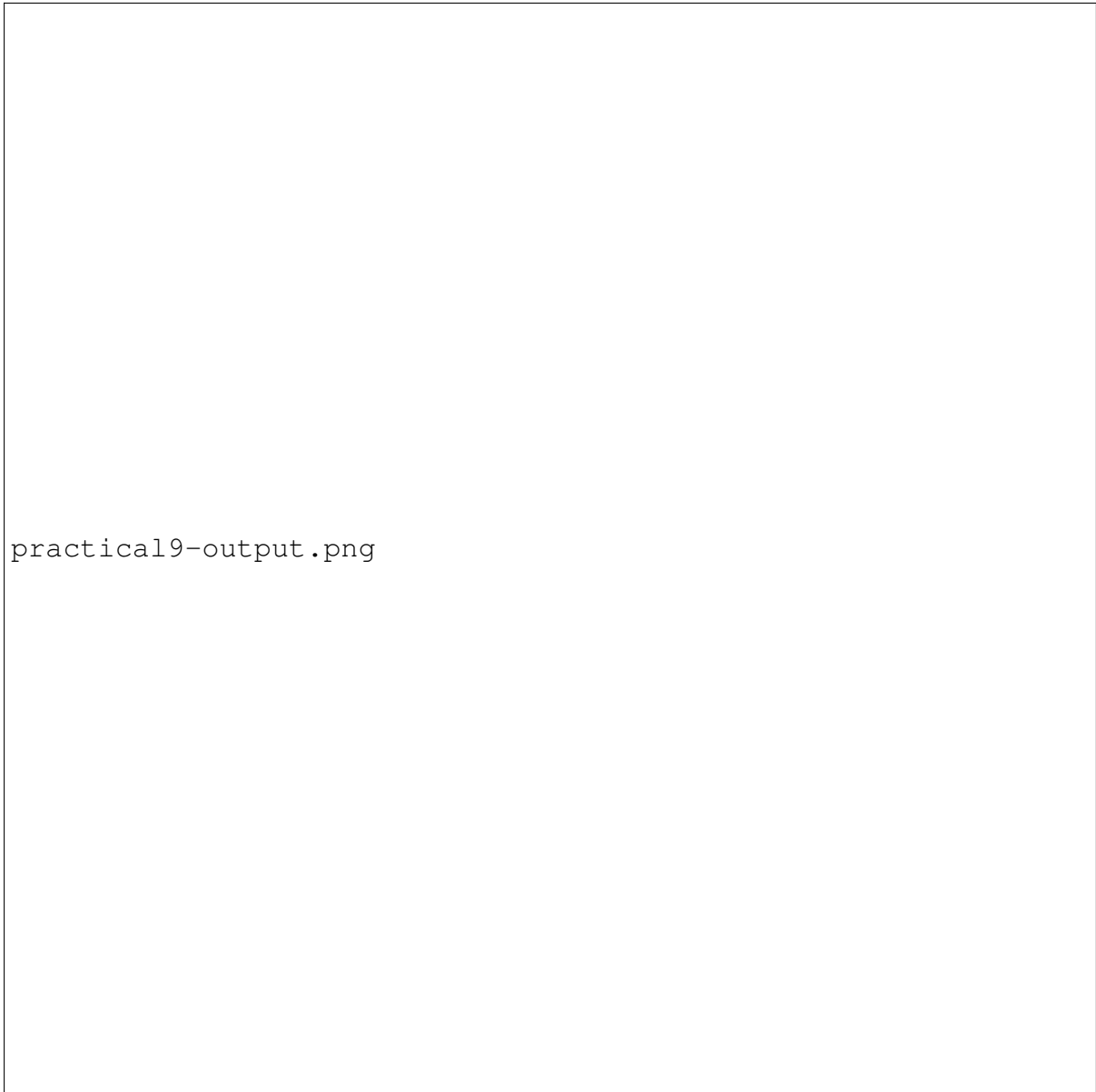


Figure 1: Output of Hill Cipher Implementation